EXOLOGY

Smart Solutions & Consulting

Autonomous AI Finance Recorder & Accounting Dashboard

Technical Documentation

Telegram Bot	Gemini 2.5 Flash	Google Sheets
Text · Voice · Image	AI Extraction Engine	Database & Dashboard

Project	Autonomous AI Finance Recorder & Accounting Dashboard
Platform	n8n + Google Sheets + Telegram + Gemini 2.5 Flash
Organization	Exology – Smart Solutions & Consulting

1. System Overview

1.1 What is This System?

Exology AI Finance Recorder is a lightweight, AI-powered financial tracking system built for small and medium business owners. It eliminates the need for complex accounting software by allowing users to record any financial transaction simply by sending a message to a Telegram bot — in Arabic or English, as text, voice, or photo.

The system automatically understands the message, extracts structured accounting data, validates it, stores it in Google Sheets, and confirms the entry — all within seconds.

1.2 Problem Being Solved

Small business owners frequently fail to record daily transactions consistently. This leads to:

- Incomplete or missing financial records
- Lack of real-time cash flow visibility
- Poor financial decision-making
- No access to basic financial reports (P&L, Cash Flow)

The Exology system reduces transaction recording to a single action: send a message.

1.3 Key Capabilities

Capability	Description
Multi-modal Input	Accepts text, voice notes, and receipt photos via Telegram
Bilingual AI	Processes Arabic and English messages natively via Gemini 2.5 Flash
Auto-categorization	Automatically maps transactions to Chart of Accounts
Data Validation	Enforces business rules before saving (amount, currency, confidence)
Financial Forecasting	Projects 30-day income, expenses, and balance from 90-day history
Health Score	Weekly AI-generated CFO analysis with actionable recommendations
Cash Drought Alert	Proactive Telegram alert when projected balance turns negative
Google Sheets DB	Stores all data in structured, accessible spreadsheets

2. Architecture Explanation

2.1 Architecture Layers

The system is structured across four distinct layers, each with a clear responsibility:

Layer	Technology	Role
Presentation	Telegram Bot	User-facing interface. Receives text, voice, and photo messages. Sends confirmation replies.
Application	n8n Workflow Engine	Orchestrates the entire pipeline — routing input, calling AI, validating data, writing to Sheets.
Intelligence	Gemini 2.5 Flash API	Understands natural language (Arabic/English), transcribes voice, reads receipts, extracts structured JSON.
Data	Google Sheets	Persistent storage for transactions, chart of accounts, forecasts, health scores, and dashboard.

2.2 System Workflows Overview

The system consists of three independent n8n workflows, each serving a distinct purpose:

Workflow	Trigger	Purpose
Exology (Main)	Telegram message received	Records financial transactions from text, voice, or photo input
Exology – Forecasting	Daily at 11:00 PM	Projects 30-day financial outlook and alerts on cash drought
Exology – Health Score	Weekly (Monday at 9:00 AM)	AI CFO analysis of last 30 days with health score and recommendations

3. Setup Instructions

3.1 Prerequisites

Before deploying the system, ensure you have the following:

- n8n instance (self-hosted or cloud) — v1.0+
- Telegram Bot Token — created via @BotFather on Telegram
- Google account with Google Sheets API enabled
- Google Gemini API Key — free tier available at aistudio.google.com
- A copy of the Exology Google Sheets database (see section 3.3)

3.2 n8n Credential Setup

1	**Telegram Bot Credential** In n8n → Credentials → New → Telegram API. Paste your Bot Token from @BotFather. Name it 'Exology Bot'.
2	**Google Sheets OAuth2 Credential** In n8n → Credentials → New → Google Sheets OAuth2. Authorize with your Google account. Name it 'Exology Sheets'.
3	**Gemini API Key** No n8n credential needed — the Gemini API key is embedded directly in the HTTP Request nodes as a URL query parameter (?key=YOUR_KEY). Replace the existing key with your own in all three workflows.

3.3 Google Sheets Database Setup

Create a new Google Spreadsheet and add the following sheets (tabs):

Sheet Name	Required Columns
Transactions	`transaction_id, timestamp, type, amount, currency, category, payment_method, description, vendor, account_code, account_name, account_type, confidence_score, input_type, raw_input`
Forecasts	`forecast_date, projected_income, predicted_expenses, predicted_balance, cash_drought, generated_at`
Health_Scores	`health_score, summary, recommendation_1, recommendation_2, recommendation_3, generated_at`

3.4 Importing the Workflows

1 **Open n8n**
Navigate to your n8n dashboard and click 'New Workflow'.

2 **Import JSON**
Click the three-dot menu (⋮) → Import from File. Import each JSON file: Exology.json, Exology - Forecasting.json, Exology - Health Score.json.

3 **Update Credentials**
After import, open each workflow and update all credential references to match your own 'Exology Bot' and 'Exology Sheets' credentials.

4 **Update Sheet ID**
Replace the Google Sheets document ID (119LsTEEgaJfmMETDke7P_0ldZBZgAx1l3l_2w-WXJq4) in all nodes with your own spreadsheet ID.

5 **Activate**
Toggle each workflow to Active. The main workflow will begin listening for Telegram messages immediately.

4. Workflow Logic

4.1 Main Workflow — Transaction Recording (Exology.json)

This is the core workflow, triggered in real-time whenever a user sends a message to the Telegram bot.

Step 1 — Telegram Trigger

The workflow starts with a Telegram Trigger node listening for incoming 'message' updates. Every message sent to the bot — regardless of type — initiates the pipeline.

Step 2 — Switch (Input Type Router)

A Switch node inspects the incoming message and routes it to one of three parallel paths based on content type:

- Text Path (Output: 'Text') — triggered when message.text is present
- Voice Path (Output: 'Voice') — triggered when message.voice.file_id is present
- Photo Path (Output: 'Photo') — triggered when message.photo[0].file_id is present

Path A — Text Input

3A	**HTTP Request → Gemini API** Sends the raw text message directly to Gemini 2.5 Flash with the Finance Extractor prompt. Returns a structured JSON object.
4A	**Code Node — Parse & Enrich** Parses the Gemini JSON response, strips any markdown fences, generates a unique transaction_id (TXN-{timestamp}), adds timestamp and input_type='text'.

Path B — Voice Input

3B	**HTTP Request → getFile** Calls the Telegram Bot API to get the file path of the voice note using its file_id.
4B	**HTTP Request → Download Audio** Downloads the actual .ogg audio file as binary data from Telegram's file server.
5B Upload	**HTTP Request → Gemini File Upload** Uploads the audio binary to Gemini's File API endpoint (uploadType=media) to obtain a hosted file URI.

5B Analysis	**HTTP Request → Gemini Transcription** Sends the file URI to Gemini 2.5 Flash with the voice extraction prompt. Gemini transcribes and extracts transaction data simultaneously.
Code JS1	**Code Node — Parse Voice Response** Parses the Gemini response, generates transaction_id, sets input_type='voice'.

Path C — Photo Input (Receipt)

3C	**HTTP Request → getFile** Retrieves the file path of the highest-resolution photo from Telegram.
4C	**HTTP Request → Download Photo** Downloads the JPEG image as binary data.
5C	**HTTP Request → Gemini Vision Upload** Uploads the image to Gemini's File API as image/jpeg.
5C Analysis	**HTTP Request → Gemini Receipt Extraction** Sends the uploaded image URI to Gemini with the receipt extraction prompt. Extracts transaction details from the visual receipt.
Code JS	**Code Node — Parse Photo Response** Parses Gemini response, generates transaction_id, sets input_type='photo'.

Merge & Save

Merge	**Merge Node** Combines the outputs of all three paths (text, voice, photo) into a single pipeline. The first data to arrive passes through.
Append Row	**Google Sheets — Append** Writes the structured transaction object to the 'Transactions' sheet with all mapped fields.
Send Message	**Telegram — Confirmation** Sends a formatted confirmation message back to the user showing the recorded transaction details.

4.2 Forecasting Workflow (Exology - Forecasting.json)

This workflow runs automatically every night at 11:00 PM to generate a 30-day financial forecast.

1	**Schedule Trigger — 11:00 PM Daily** Fires every night at 23:00. No user action required.
2	**Get Transactions** Reads all rows from the Transactions sheet.
3	**Code — Calculate Averages** Filters transactions from the last 90 days. Calculates total income and total expenses. Derives average daily income and average daily expense. Projects these forward for each of the next 30 days, generating a cumulative forecast array (30 rows).
4	**Append to Forecasts Sheet** Saves all 30 forecast rows to the Forecasts sheet with: forecast_date, projected_income, predicted_expenses, predicted_balance, cash_drought flag, generated_at.
5	**If — Cash Drought Check** Evaluates the cash_drought field. If any projected day shows a negative balance (cash_drought = true), the 'true' branch activates.
6	**Code JS2 — Extract Day 30 Summary** Isolates the final day (day 30) as the worst-case scenario for the alert message.
7	**Telegram Alert** Sends a cash drought warning to the user's Telegram chat ID with projected totals and the expected negative balance date.

4.3 Health Score Workflow (Exology - Health Score.json)

This workflow runs every Monday at 9:00 AM, generating a CFO-style health analysis.

1	**Schedule Trigger — Weekly Monday 9:00 AM** Fires once per week.
2	**Get Transactions** Reads all rows from the Transactions sheet.
3	**Code — 30-Day Summary** Filters to the last 30 days. Calculates: total income, total expenses, net balance, expense-to-income ratio, transaction count, and top spending categories.
4	**HTTP Request → Gemini CFO Prompt** Sends the financial summary to Gemini with the AI CFO prompt. Requests a JSON response containing: health_score ('Healthy' / 'Warning' / 'Critical'), a 2-sentence summary, and 3 actionable recommendations.

5	**Code JS1 — Parse Health Response** Extracts the JSON from Gemini's response using regex, parses it, adds generated_at timestamp.
6	**Append to Health_Scores Sheet** Saves the health report to Google Sheets for historical tracking.
7	**Telegram Weekly Report** Sends a formatted weekly financial health report to the user via Telegram, including the health score, summary, and all three recommendations.

5. AI Prompts

5.1 Transaction Extraction Prompt (Text)

Used in: Main Workflow — Node 3A

```
You are a financial data extractor for a business. Extract transaction details
from this message and return ONLY a valid JSON object with no markdown,
no explanation, no code fences. Use exactly these fields:

{
  "amount": number,
  "type": "income" or "expense",
  "category": string,
  "description": string,
  "payment_method": "cash" or "card" or "transfer" or "unknown",
  "vendor": string or "unknown",
  "currency": "EGP" or "USD",
  "confidence_score": number between 0 and 1
}

Message to analyze: {{ $json.message.text }}
```

5.2 Voice Transaction Extraction Prompt

Used in: Main Workflow — Node 5B analysis

```
This is a voice message recording a business transaction.
Transcribe it and extract the transaction details.
Return ONLY a valid JSON object with no markdown:
{
  "amount": number,
  "type": "income" or "expense",
  "category": string,
  "description": string,
  "payment_method": "cash" or "card" or "transfer" or "unknown",
  "vendor": string or "unknown",
  "currency": "EGP" or "USD",
  "confidence_score": number between 0 and 1
}
```

5.3 Health Score (AI CFO) Prompt

Used in: Health Score Workflow — HTTP Request node

```
You are an AI CFO analyzing a small business financial health.
Based on this 30-day summary, return ONLY a valid JSON object with no markdown:
{
  "health_score": "Healthy" or "Warning" or "Critical",
  "summary": string (2 sentences max),
  "recommendation_1": string,
  "recommendation_2": string,
  "recommendation_3": string
}

Financial data:
  Period: {{ $json.period }}
  Total Income: {{ $json.total_income }} EGP
  Total Expenses: {{ $json.total_expense }} EGP
  Net Balance: {{ $json.net_balance }} EGP
  Expense to Income Ratio: {{ $json.expense_to_income_ratio }}
  Transaction Count: {{ $json.transaction_count }}
```

5.4 Prompt Design Principles

All prompts in this system follow a consistent set of rules:

- JSON-only output — Every prompt explicitly forbids markdown, code fences, and explanations to ensure reliable parsing
- Strict field definitions — Each field type and allowed value is explicitly listed to reduce ambiguity
- Bilingual by default — Gemini 2.5 Flash handles Arabic and English natively without explicit instruction
- Confidence scoring — Every transaction extraction returns a confidence_score (0–1) used for validation gating

6. Data Validation Rules

The system enforces the following validation rules on every extracted transaction before it is saved:

Scenario	Action	Logic
Missing amount	**Reject**	Cannot record without a monetary value
Missing date/timestamp	**Use current date/time**	System auto-fills using new Date().toISOString()
Missing currency	**Default to EGP**	Egyptian Pound assumed if not specified
Low confidence (< 0.6)	**Flag for review**	Transaction saved with Pending status; user alerted
AI response parse error	**Return error object**	{ error: true, raw_response } saved; user notified
Missing JSON structure	**Regex fallback**	Regex /{[\s\S]*}/ attempts to extract embedded JSON

7. Limitations

7.1 Technical Limitations

- No real-time user confirmation step — Transactions are saved immediately after AI extraction without a 'confirm before saving' prompt in the current implementation. The proposal described this step, but it requires a stateful conversation flow not yet implemented.
- Gemini API rate limits — Free tier Gemini API has limited requests per minute. High-volume usage may cause delays or failures.
- Voice note format — Only .ogg format (Telegram default) is handled. MP3, WAV, or other formats are not tested.
- Photo quality dependency — Receipt extraction accuracy depends heavily on image clarity, lighting, and font readability.
- No duplicate detection — The system does not check if the same transaction has already been recorded before saving.
- Google Sheets row limit — Google Sheets supports up to 10 million cells. For very active businesses, the Transactions sheet may require periodic archiving.

7.2 AI Limitations

- Confidence score reliability — The confidence_score is self-reported by the AI and may not always accurately reflect extraction quality.
- Ambiguous messages — Vague inputs like 'paid some money' will result in low confidence and incomplete data.
- Currency assumption — The system defaults to EGP. Multi-currency transactions within a single message are not supported.
- Category consistency — Without a fixed category list enforced at the prompt level, the AI may use slightly different category names for similar expenses (e.g., 'Internet' vs. 'Connectivity').

7.3 Infrastructure Limitations

- No offline capability — All processing requires active internet connectivity.
- n8n dependency — If the n8n instance goes offline, no transactions can be recorded until it is restored.
- Single user bot — The current implementation uses a hardcoded Telegram chat ID. Multi-user support would require user registration and routing logic.
- No audit trail — There is currently no mechanism to edit or delete recorded transactions. Any corrections require direct Sheets access.

8. Future Improvements

8.1 Short-Term (Next Sprint)

- Confirmation step before saving — Add a Telegram inline keyboard (Yes / Edit / Cancel) after extraction, requiring user approval before the transaction is written to Sheets.
- Fixed category taxonomy — Define a standard Chart of Accounts category list in a lookup sheet and enforce it at the AI prompt level for consistency.
- Duplicate detection — Hash each incoming message and compare against existing transactions to prevent accidental double-recording.
- Transaction editing via bot — Allow users to reply to a confirmation message with corrections that update the existing row.

8.2 Medium-Term

- Multi-user support — Implement user registration flow, storing each Telegram user_id with their associated Sheets database. Route all data accordingly.
- PDF & document receipts — Extend the photo path to handle PDF attachments using Gemini's document understanding capability.
- Monthly financial statements — Auto-generate Income Statement and Cash Flow Summary as formatted Google Sheets reports, emailed or sent via Telegram monthly.
- Smart categorization learning — Log user corrections to category assignments and use them to fine-tune prompt instructions over time.

8.3 Long-Term

- Web dashboard — Build a lightweight web interface connected to the same Google Sheets database for richer visualization (charts, filters, exports).
- Budget tracking — Allow users to set monthly budgets per category and receive alerts when approaching limits.
- Accounting integrations — Connect to QuickBooks, Xero, or Wave for professional-grade accounting output.
- Voice command shortcuts — Recognize shortcut phrases ('quick expense', 'log income') to reduce AI processing overhead for common transaction types.
- Mobile app — A dedicated mobile app wrapping the Telegram bot functionality with a native UI for non-technical users.

9. Google Sheets Database Structure

9.1 Transactions Sheet — Field Reference

Field	Type	Description
transaction_id	String	Unique ID auto-generated: TXN-{Unix timestamp}
timestamp	ISO 8601	Date and time of recording (not necessarily transaction date)
type	Enum	'income' or 'expense'
amount	Number	Transaction value (positive)
currency	Enum	'EGP' or 'USD'
category	String	Business category (e.g., Internet, Rent, Sales)
payment_method	Enum	'cash', 'card', 'transfer', or 'unknown'
description	String	Short natural language description from AI
vendor	String	Merchant/vendor name or 'unknown'
account_code	String	Chart of Accounts reference code
account_name	String	Full account name
account_type	Enum	'Expense' or 'Revenue'
confidence_score	Float 0–1	AI self-reported extraction confidence
input_type	Enum	'text', 'voice', or 'photo'
raw_input	String	Original message text or 'voice_message' / 'photo'

9.2 Forecasting Logic (Code Reference)

The forecasting algorithm implemented in JavaScript inside n8n:

```
// Filter last 90 days of transactions
const recent = rows.filter(r => (now - new Date(r.json.timestamp)) < 90_days);

// Sum income and expenses
recent.forEach(r => {
  if (r.json.type === "income") totalIncome += amount;
  if (r.json.type === "expense") totalExpense += amount;
});

// Calculate daily averages
```

```
const avgDailyIncome = totalIncome / 90;
const avgDailyExpense = totalExpense / 90;
const avgDailyNet = avgDailyIncome - avgDailyExpense;

// Project forward 30 days (cumulative)
for (let i = 1; i <= 30; i++) {
  projected_income: avgDailyIncome * i,
  projected_expense: avgDailyExpense * i,
  projected_balance: avgDailyNet * i,
  cash_drought: avgDailyNet < 0   // true if daily net is negative
}
```